

Extending the Reach of Testing

Gregory S. DeLozier, PhD

A Python function is only one place where software does its work. A program may get data from a service, read a page, or respond to someone clicking a control. We need ways to examine those behaviors too.

Here we will work with existing public systems: GitHub's API and Wikipedia. We will retrieve information, decode a page, and interact with a browser. The aim is to understand what each tool lets us observe. We are not building an application in this chapter.

Three different jobs

An **application programming interface**, or **API**, lets one program request services or data from another. Requests sends requests using the **Hypertext Transfer Protocol**, or **HTTP**. It can retrieve API data or the text of a web page (1).

A page's **Hypertext Markup Language**, or **HTML**, describes its content and structure. BeautifulSoup parses that text into elements we can examine (2). Requests gets the page. BeautifulSoup decodes its structure. Neither runs the page's JavaScript.

WebDriver is a browser automation standard. Selenium provides a Python interface; ChromeDriver implements browser control for Google Chrome (3). Chrome runs the JavaScript, so our code can type into a search box and inspect content that appears afterward.

Job	Tool	What we obtain
Get repository data from GitHub	Requests	An HTTP response with structured data
Get Wikipedia's page HTML	Requests	The text delivered by the server
Decode that HTML	Beautiful Soup	Headings, paragraphs, links, and attributes
Type into Wikipedia's search and select a suggestion	WebDriver through ChromeDriver	Interactive behavior and dynamically displayed content

These jobs overlap, but they are not interchangeable. A browser can read a static page, although starting one may be unnecessary. Finding a search box in downloaded HTML does not establish that typing into it produces suggestions.

Sidebar: installing on macOS and Windows

Install Python 3 and Google Chrome. Open a terminal in this chapter's code directory. On macOS:

```
python3 -m venv .venv
source .venv/bin/activate
```

On Windows, in Command Prompt:

```
py -m venv .venv
.venv\Scripts\activate.bat
```

For PowerShell, activation is `.\.venv\Scripts\Activate.ps1`. If local policy blocks it, use Command Prompt or invoke `.\.venv\Scripts\python.exe` directly instead of `python`.

After activation, install the packages:

```
python -m pip install -r requirements.txt
```

`beautifulsoup4` is the package name; `bs4` is the import. Selenium includes Selenium Manager. When `webdriver.Chrome()` needs a driver, the manager can find the installed Chrome version and download a matching `ChromeDriver`. The first run may need internet access (4).

Activate the environment in each new terminal. Installation and execution must use the same Python environment.

Sidebar: Linux, for completeness

Use the macOS environment commands. Some distributions need a separate system package for Python's `venv` support. Install the Python requirements and a compatible Chrome or Chromium browser with its system dependencies.

With no graphical desktop, use headless mode: create `options = webdriver.ChromeOptions()`, call `options.add_argument("--headless=new")`, and pass `options=options` to `webdriver.Chrome`. The browser still executes JavaScript; it simply has no visible window.

Each example is a `test_` function with assertions, in a file whose name starts with `test_`. Pytest discovers those functions and reports whether they pass (5). From the chapter's code folder, run:

```
python -m pytest -v -s
```

`-v` shows each test's name and result. `-s` leaves printed output visible during the demonstration (6). Printing helps us inspect a response; assertions decide whether the test passes. Each file also retains a `main` function for direct execution. Run without Python's `-O` option, which disables ordinary assertions.

Requests: getting API data

GitHub exposes information about public repositories through its API. We will request the `python/cpython` repository, which contains the Python implementation's source. Reading this public endpoint does not require a login or token (7).

The response uses **JavaScript Object Notation**, or **JSON**, a text format for structured data. Requests can decode it into Python dictionaries and lists with `response.json()` (1).

```
import requests

URL = "https://api.github.com/repos/python/cpython"
HEADERS = {"Accept": "application/vnd.github+json",
           "User-Agent": "SoftwareEngineeringCourse/1.0"}

def test_repository_data():
    response = requests.get(URL, headers=HEADERS, timeout=15)
    assert response.status_code == 200, response.status_code
    repo = response.json()
    assert repo["full_name"] == "python/cpython"
    assert repo["owner"]["login"] == "python"
    assert repo["private"] is False
    print(repo["full_name"], repo["description"])
```

Run `python -m pytest -v -s test_api.py`. A status of 200 says that the request succeeded. The other assertions check which repository came back and whether it is public. A successful response containing the wrong repository would not pass those checks.

The timeout prevents an indefinite wait for network activity; it is not a guaranteed total running time. If the request fails, inspect the status before assuming the returned data is wrong. GitHub can limit requests, and a classroom may share one public network address. Keep the exercise to a few requests rather than repeatedly polling.

Live exercise: Print `repo.keys()`. Find the repository's description and default branch. Then examine its star count. Which values can we reasonably check against an exact expectation, and which are likely to change between classes?

Requests: getting static page content

The same library can retrieve Wikipedia's HTML. This is still retrieval, not browser automation:

```
import requests

URL = "https://en.wikipedia.org/wiki/Selenium_(software)"
HEADERS = {"User-Agent": "SoftwareEngineeringCourse/1.0 (educational example)"}

def test_wikipedia_response():
    response = requests.get(URL, headers=HEADERS, timeout=15)
    assert response.status_code == 200, response.status_code
    assert "text/html" in response.headers["Content-Type"]
    assert "Selenium_WebDriver" in response.text
    print("HTTP status:", response.status_code)
    print(response.text[:500])
```

Run `python -m pytest -v -s test_wikipedia_response.py` to print the status and the first 500 characters of the returned HTML.

This article includes a section about Selenium WebDriver (8). The server sends its text as part of the HTML. Requests does not need to run JavaScript to get it.

Here, “static content” means the content delivered in the response. Wikipedia can also have interactive features, and its articles can change. We are examining the delivered document before any browser interaction.

Live exercise: Search the returned text for `Selenium_WebDriver`. What surrounds it? Does finding that string establish that we have the section’s explanatory paragraph?

Beautiful Soup: decoding the page

A string search cannot reliably distinguish a heading from a navigation link or a script. Beautiful Soup parses the HTML so we can ask about specific elements (2).

```
import requests
from bs4 import BeautifulSoup

URL = "https://en.wikipedia.org/wiki/Selenium_(software)"
HEADERS = {"User-Agent": "SoftwareEngineeringCourse/1.0 (educational example)"}

def test_wikipedia_html():
    response = requests.get(URL, headers=HEADERS, timeout=15)
    assert response.status_code == 200, response.status_code
    soup = BeautifulSoup(response.text, "html.parser")
    title = soup.find("h1")
    assert title is not None
    assert title.get_text(strip=True) == "Selenium (software)"
    heading = soup.find(id="Selenium_WebDriver")
    assert heading is not None
    paragraph = heading.find_next("p")
    assert paragraph is not None
    text = paragraph.get_text(" ", strip=True)
    assert "WebDriver" in text
    print(text)
    link = paragraph.find("a")
    assert link is not None
    print("First link:", link.get_text(" ", strip=True))
    print("Destination:", link.get("href"))
```

Run `python -m pytest -v -s test_static_html.py`. The first two lines inside the test retrieve the document. `BeautifulSoup(...)` begins the decoding step. `find("h1")` locates the main heading; `find(id="Selenium_WebDriver")` locates the section heading. `find_next("p")` gets the following paragraph. `get_text(" ", strip=True)` combines its text while keeping a space between text fragments.

Check that an element exists before reading it. Otherwise an absent heading can become an unhelpful error about calling a method on `None`.

This example reads the first paragraph about WebDriver. It does not claim to validate every fact in the article. It also depends on Wikipedia’s current document structure. A changed heading identifier could break the lookup even though the article still looks fine to a reader.

Live exercise: The script prints the first link inside the paragraph and its href attribute. Does the address stand alone, or is it relative to Wikipedia? Change the code to inspect a different link. What else would we have to do to check its destination?

ChromeDriver: interactive behavior and dynamic pages

Now open Wikipedia in a browser and type “Selenium” into its search box. Suggestions appear while we type. Select “Selenium (software)” and read its WebDriver section.

Downloading the initial HTML does not perform that interaction. Chrome must run the page’s JavaScript and display the suggestions. Selenium sends commands through ChromeDriver, then reads the browser’s current **Document Object Model**, or **DOM**, which represents the page as it now exists (3).



The helper below looks through the current suggestions. It returns the desired link when it appears; otherwise it returns False, allowing the wait to check again.

```
def software_suggestion(browser):
    for item in browser.find_elements(By.CSS_SELECTOR, ".cdx-menu-item"):
        if item.is_displayed():
            label = item.text.split("\n", 1)[0]
            if label == "Selenium (software)":
                return item.find_element(By.TAG_NAME, "a")
    return False
```

.cdx-menu-item is a **Cascading Style Sheets**, or **CSS**, class selector. It identifies Wikipedia’s current suggestion items. That class name belongs to this site’s implementation and may change.

The browser test uses that helper:

```
def test_wikipedia_search():
    browser = webdriver.Chrome()
    try:
        browser.set_window_size(1280, 900)
        browser.set_page_load_timeout(30)
        browser.get("https://en.wikipedia.org/wiki/Main_Page")
        wait = WebDriverWait(browser, 15)
        search = wait.until(EC.element_to_be_clickable((By.NAME, "search")))
        search.send_keys("Selenium")
        suggestion = wait.until(software_suggestion)
        suggestion.click()
        wait.until(EC.text_to_be_present_in_element(
            (By.ID, "firstHeading"), "Selenium (software)"))
        assert browser.find_element(By.ID, "firstHeading").text == "Selenium (software)"
        heading = wait.until(EC.presence_of_element_located(
            (By.ID, "Selenium_WebDriver")))
        paragraph = heading.find_element(By.XPATH, "following::p[1]")
        text = paragraph.text.strip()
        assert "WebDriver" in text
        print(text)
    finally:
        browser.quit()
```

Run `python -m pytest -v -s test_wikipedia_webdriver.py`. The complete file includes the Selenium imports and a main function. It opens Chrome, performs the search, prints the paragraph, and closes the browser even if an assertion fails.

`WebDriverWait` repeatedly checks a condition until it succeeds or reaches the timeout (9). The search control must be ready for interaction. The desired suggestion must appear. After the click, the article heading must contain the expected title. Each wait corresponds to a particular stage in the interaction.

The paragraph lookup uses **XML Path Language**, or **XPath**; XML stands for Extensible Markup Language. `following::p[1]` selects the first paragraph after the section heading in document order. Here the article body itself was available in the static HTML too. The dynamic part of this exercise was the search suggestion, not a claim that every word on Wikipedia requires JavaScript.

Live exercise: Begin with opening the page and typing the query. Inspect the suggestions before adding the click. Then add the article-title check and paragraph lookup. Change the expected title temporarily and observe which condition fails.

What does each observation establish?

Observation	What it supports	What it does not establish
GitHub returns the expected repository fields	The API returned the data we checked	GitHub's website displays it correctly
Wikipedia HTML contains the expected paragraph	The delivered document contains that text	Search suggestions work
Typing produces the desired suggestion	That interactive path worked on this run	Every query or browser works
Selecting the suggestion opens the expected article	The checked navigation succeeded	Every statement in the article is correct

We are checking public systems that we do not control. A failure may come from our code, the network, a rate limit, or a changed page. Read the failure and investigate its cause. An external service being unavailable is not automatically evidence that our own program has a logic error.

The useful habit is to connect an observation to a clear claim. A tool gets us access; we still have to decide what that access can tell us.

A final note about waiting

ChromeDriver is a capable implementation of the WebDriver standard (3). Timing can still be awkward: the page may have loaded while a later update is unfinished. Selenium provides explicit waits, but we choose what to wait for and how long to allow (9). Checking too early can make working behavior appear broken.

Playwright handles more of this waiting automatically. It waits for controls to be ready before acting, and its retrying assertions wait for the expected state. This can reduce timing problems and explicit wait code, although the expected outcome and timeout still matter (10). The reference below is a starting point for learning about it.

References

1. Requests project. "Quickstart." <https://requests.readthedocs.io/en/latest/user/quickstart/>
2. Leonard Richardson. "Beautiful Soup Documentation." <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
3. Selenium project. "WebDriver." <https://www.selenium.dev/documentation/webdriver/>
4. Selenium project. "Selenium Manager." https://www.selenium.dev/documentation/selenium_manager/
5. pytest developers. "Good Integration Practices," test discovery conventions. <https://docs.pytest.org/en/stable/explanation/goodpractices.html>
6. pytest developers. "How to capture stdout/stderr output." <https://docs.pytest.org/en/stable/how-to/capture-stdout-stderr.html>
7. GitHub. "Get a repository." REST API documentation. <https://docs.github.com/en/rest/repos/repos#get-a-repository>

8. Wikipedia contributors. “Selenium (software),” Selenium WebDriver section. [https://en.wikipedia.org/wiki/Selenium_\(software\)#Selenium_WebDriver](https://en.wikipedia.org/wiki/Selenium_(software)#Selenium_WebDriver)
9. Selenium project. “Waiting Strategies.” <https://www.selenium.dev/documentation/webdriver/waits/>
10. Microsoft. “Auto-waiting.” Playwright for Python. <https://playwright.dev/python/docs/actionability>

Questions for Thought

1. GitHub returns status 200, but the repository name is wrong. What did the status establish, and what did it leave unanswered?
2. Which repository fields make useful exact checks? Which would make a test fail simply because normal activity continued?
3. Requests retrieves the WebDriver paragraph successfully, but the browser search fails. Why are those results compatible?
4. Wikipedia changes a heading’s identifier while leaving its visible wording alone. Which examples might fail? What would you inspect before changing the test?
5. Why is a fixed five-second sleep a poor substitute for waiting for the desired search suggestion? What decision remains even with a condition-based wait?